
Per-site retrieval of lunar regolith thermal conductivity K_d at the Apollo 15 & 17 heat-flow boreholes

The physics, the code, and the method — explained from the ground up

A complete companion manuscript to the JGR letter.

R. P. Gregorio • Kasai Laboratory, Institute of Science Tokyo

Abstract

We measure how well heat conducts through the deep lunar regolith — the parameter K_d — separately at the Apollo 15 and Apollo 17 heat-flow boreholes, and find them to differ by about a factor of two ($K_{d,A15}^* = 4.58$, $K_{d,A17}^* = 8.12 \text{ mW m}^{-1} \text{ K}^{-1}$). This document explains *everything* behind that sentence, assuming no prior knowledge: what heat conduction is and how we model it; the two published conductivity models and how to reproduce them; the numerical solver line by line; the subtle bug we found in reaching steady state and the faster, self-certifying method that replaced it; how K_d is actually retrieved from the data; how the uncertainty is quantified; why the basal heat flux Q_b matters; and why the two sites plausibly differ. Figures and code listings accompany every step. Sections needed for the oral presentation are marked with a ★.

★ FOR TOMORROW'S TALK

If you only prepare a few things, prepare these (each is one slide):

1. The question & the two sites — §1.1, Fig. 1.1.
2. What K_d is and the conductivity models — §3.1, Fig. 3.1.
3. Why we need steady state — §5.1.
4. Old vs. new solver (the GIFs) — §5.2, Figs. 5.1, 5.2.
5. They give the same answer — §5.4, Fig. 5.4.
6. The retrieval & the result — §6, Figs. 6.1, 6.2.
7. K_d depends on Q_b — §8, Fig. 8.1.
8. Why the sites differ — §9.

Contents

1	What we are trying to do, and why	3
1.1	The question	3
1.2	Why it matters	3
1.3	The shape of the whole argument	4
2	The physics, from the ground up	5
2.1	Heat conduction and the heat equation	5
2.2	The lunar forcing: the daily wave and the skin depth	5
2.3	Boundary conditions: top and bottom	6
2.4	Periodic steady state and flux closure	6
3	The two conductivity models (and how to reproduce them)	7
3.1	Hayne et al. (2017): the H-parameter model	7
3.2	Martínez & Siegler (2021): a density-based model	7
4	The numerical solver, line by line	9
4.1	The geometric depth grid	9
4.2	One time step: Crank–Nicolson	9
4.3	Solving the system: the Thomas algorithm	10
4.4	The non-linear surface: Newton’s method	10
4.5	Stepping through a lunation: the spin-up loop	10
5	Reaching steady state: the old method, the bug, and the fix	11
5.1	Why we even need steady state	11
5.2	The old method and the bug we found (flag F1)	11
5.3	The fix: a flux-anchored shortcut	12
5.4	They give the same answer	13
5.5	It certifies itself	14
6	Retrieving K_d from the data	15
6.1	One forward run: <code>run_with</code>	15
6.2	The sweep and the minimum	15
6.3	The result	16
7	How sure are we? (uncertainty)	17
7.1	The bootstrap	17
7.2	Bayesian cross-check and model selection	17
8	The role of the basal flux Q_b	19
9	Why might the two sites differ?	20
10	Independent cross-checks	21
11	How the code is organised, and a C++ future	22
12	Conclusion: how to defend every claim	23

Chapter 1

What we are trying to do, and why

1.1 The question

Every airless body loses its internal heat through a blanket of broken rock — *regolith*. How fast that heat escapes, and how steeply temperature rises with depth, is controlled by the regolith's **thermal conductivity**: a number, K , that says how many watts of heat flow through a square metre for each degree of temperature change per metre of depth (units $\text{W m}^{-1} \text{K}^{-1}$). Deep down, that number settles to a value we call K_d (“ d ” for deep).

The Apollo 15 and 17 missions buried temperature probes in the Moon (the Heat-Flow Experiment, HFE) and recorded subsurface temperatures from 1971 to 1977. The standard practice is to assume *one* global K_d for the whole Moon. **Our question is simply: what is K_d at each of the two boreholes, separately — and do they differ?** (Fig. 1.1.)

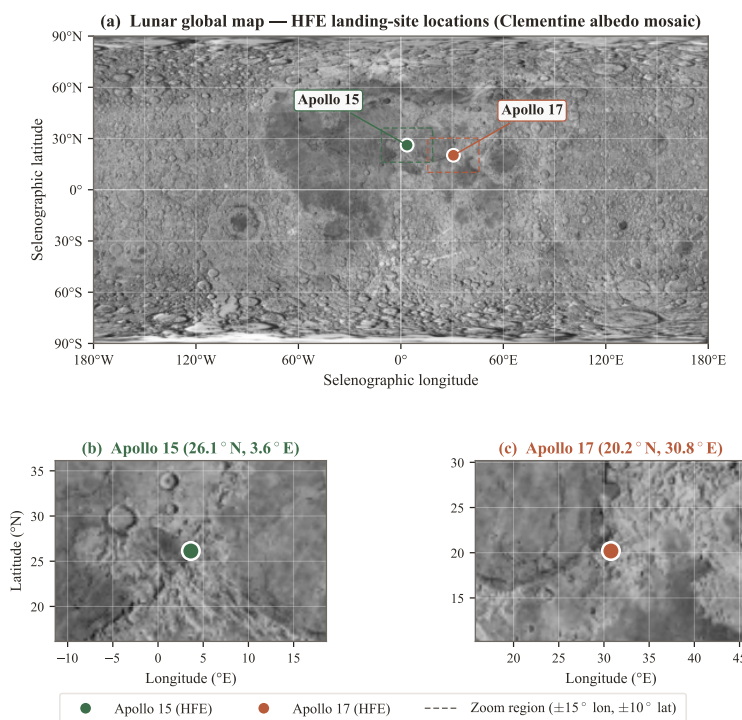


Figure 1.1: The two heat-flow boreholes. Apollo 15 sits at Hadley Rille on the Imbrium mare/Apennine boundary; Apollo 17 in the Taurus–Littrow highland valley. Different terrains → possibly different regolith.

1.2 Why it matters

K_d sets the deep temperature, which feeds models of subsurface ice stability, future drilling and habitat thermal design, and the Moon’s heat budget. A wrong, one-size-fits-all K_d propagates into all of them. Getting it right *per site* is the contribution.

1.3 The shape of the whole argument

In one breath: *(i)* heat conduction is described by one equation; *(ii)* two published formulas give K as a function of temperature and depth; *(iii)* we put one of them in a computer model of the regolith and run it until it stops changing (“steady state”); *(iv)* we ask which K_d makes the model’s deep temperatures match the Apollo sensors; *(v)* the hard part was reaching steady state correctly — the bulk of this document — and *(vi)* we then quantify the uncertainty and interpret the result. Each clause is a chapter.

Chapter 2

The physics, from the ground up

2.1 Heat conduction and the heat equation

Heat flows from hot to cold. Fourier’s law says the heat flux q (W m^{-2}) through a material is proportional to how steeply temperature changes with position:

$$q = -K \frac{\partial T}{\partial z}, \quad (2.1)$$

where z is depth and K the conductivity. The minus sign says heat flows *downhill* in temperature. Now demand energy conservation in a thin slab: the rate at which it stores heat equals what flows in minus what flows out. That gives the **heat equation**,

$$\rho(z) c_p(T) \frac{\partial T}{\partial t} = \frac{\partial}{\partial z} \left(K(T, z) \frac{\partial T}{\partial z} \right), \quad (2.2)$$

where ρ is density (kg m^{-3}) and c_p the specific heat ($\text{J kg}^{-1} \text{K}^{-1}$): the left side is “heat stored,” the right side is “net heat conducted in.” This single equation is the physics of the entire study (Fig. 2.1).

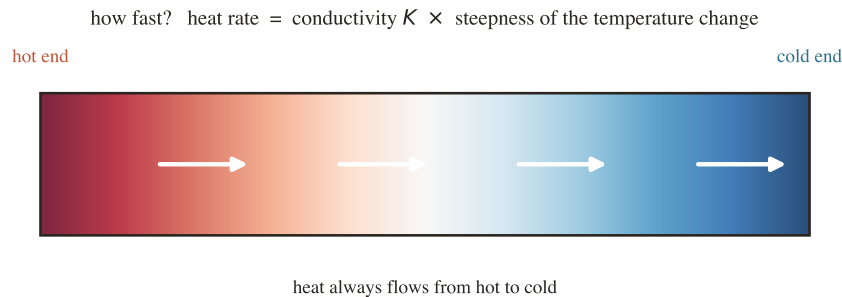


Figure 2.1: Heat conduction in a column: flux $q = -K \partial_z T$ moves heat down the temperature gradient; the heat equation is the bookkeeping of what each layer stores versus passes on.

In plain words

Think of the regolith as a stack of thin layers. Each layer can hold heat (that’s ρc_p) and can pass heat to its neighbours (that’s K). The heat equation just says: a layer warms up if more heat arrives than leaves. Solving it means tracking that exchange through time and depth.

2.2 The lunar forcing: the daily wave and the skin depth

At the surface, the Sun drives a huge day–night temperature swing (from ~ 100 K at night to ~ 390 K at noon). This “thermal wave” travels downward but *dies out* quickly: its amplitude falls by $1/e$ over the **skin depth** $\delta = \sqrt{2\kappa/\omega}$, where $\kappa = K/(\rho c_p)$ is the thermal diffusivity and $\omega = 2\pi/P$ the angular frequency of the lunar day. For the Moon $\delta \approx 10$ – 20 cm. Below a few skin depths, the regolith barely feels day or night — it responds only to the *average* heating. This split (a fast, shallow daily wave on top of a slow, deep mean) is the single most important fact for everything that follows.

2.3 Boundary conditions: top and bottom

Equation (2.2) needs a rule at each end of the column.

- **Top (surface): a radiative energy balance.** The surface has no contact with anything above it, so sunlight absorbed must equal heat radiated away plus heat conducted down:

$$(1 - A) S(t) = \varepsilon \sigma T_s^4 + K \left. \frac{\partial T}{\partial z} \right|_0, \quad (2.3)$$

with A the albedo (reflectivity), $S(t)$ the incident sunlight, ε the emissivity, and σ the Stefan–Boltzmann constant. The T_s^4 makes this *non-linear* — it must be solved numerically each timestep.

- **Bottom: the geothermal flux.** From far below, the Moon’s interior supplies a small, steady heat flux Q_b (about 15–21 mW m^{−2}). So at the base of the column, $K \partial_z T = Q_b$.

2.4 Periodic steady state and flux closure

After long enough, the column reaches a **periodic steady state**: every lunation is identical to the last. Two conditions define it, and we will lean on the second one constantly:

1. the surface skin repeats its daily cycle;
2. below the skin there are no heat sources, so in steady state the same geothermal flux passes through *every* layer. With Fourier’s law this fixes the deep gradient of the cycle-mean temperature $\langle T \rangle$:

$$\boxed{K(\langle T \rangle, z) \frac{d\langle T \rangle}{dz} = Q_b \quad \implies \quad \frac{d\langle T \rangle}{dz} = \frac{Q_b}{K(\langle T \rangle, z)}} \quad (2.4)$$

Key point

Equation (2.4) is the backbone of the whole method. It says the deep temperature profile is *not* a free thing to be discovered — once you know the temperature at one depth, energy conservation fixes the slope everywhere below. Remember it; Chapters 5 and 6 are built on it.

Chapter 3

The two conductivity models (and how to reproduce them)

K is not a single number — it depends on temperature (hotter regolith radiates across pores, conducting better) and depth (deeper regolith is compacted, conducting better). Two published models capture this.

3.1 Hayne et al. (2017): the H-parameter model

The solid “contact” conductivity rises from a surface value K_s to a deep value K_d as the regolith compacts, over an e -folding depth H ; a radiative term $\propto T^3$ is added on top:

$$K_c(z) = K_d - (K_d - K_s) e^{-z/H}, \quad (3.1)$$

$$K(T, z) = K_c(z) \left[1 + \chi (T/T_{\text{ref}})^3 \right]. \quad (3.2)$$

In code this is four lines (`lunar/properties.py`):

```
def conductivity_hayne(T, z, Ks=7.4e-4, Kd=3.4e-3, H=0.06, chi=2.7):  
    Kc = Kd - (Kd - Ks) * np.exp(-z / H) # contact part: K_s → K_d  
    return Kc * (1.0 + chi * (T / 350.0) ^3) # + radiative T^3 term
```

The published parameters: $K_s = 0.74$, $K_d = 3.4 \text{ mW m}^{-1} \text{ K}^{-1}$; $H = 6 \text{ cm}$; $\chi = 2.7$; $T_{\text{ref}} = 350 \text{ K}$. The density follows the same shape, $\rho(z) = \rho_d - (\rho_d - \rho_s) e^{-z/H}$ with $\rho_s = 1100$, $\rho_d = 1800 \text{ kg m}^{-3}$.

Why the literature K_d is 3.4. Hayne et al. fit Eqs. (3.1)–(3.2) to the *globally averaged* day–night temperature curves measured from orbit by the Diviner instrument. A single set of parameters reproduced the global average, and $K_d = 3.4$ is that global deep asymptote. It was never site-specific — which is the gap we fill (Fig. 3.1a).

3.2 Martínez & Siegler (2021): a density-based model

This writes K directly in terms of bulk density ρ :

$$K(T, \rho) = (A_1 \rho + A_2) k_{\text{am}}(T) + (B_1 \rho + B_2) T^3, \quad (3.3)$$

with $k_{\text{am}}(T)$ a fitted amorphous-solid polynomial. Same idea, different knobs: where Hayne’s free parameter is K_d , Martínez’s is effectively the deep density. We use it as an independent cross-check (Ch. 10).

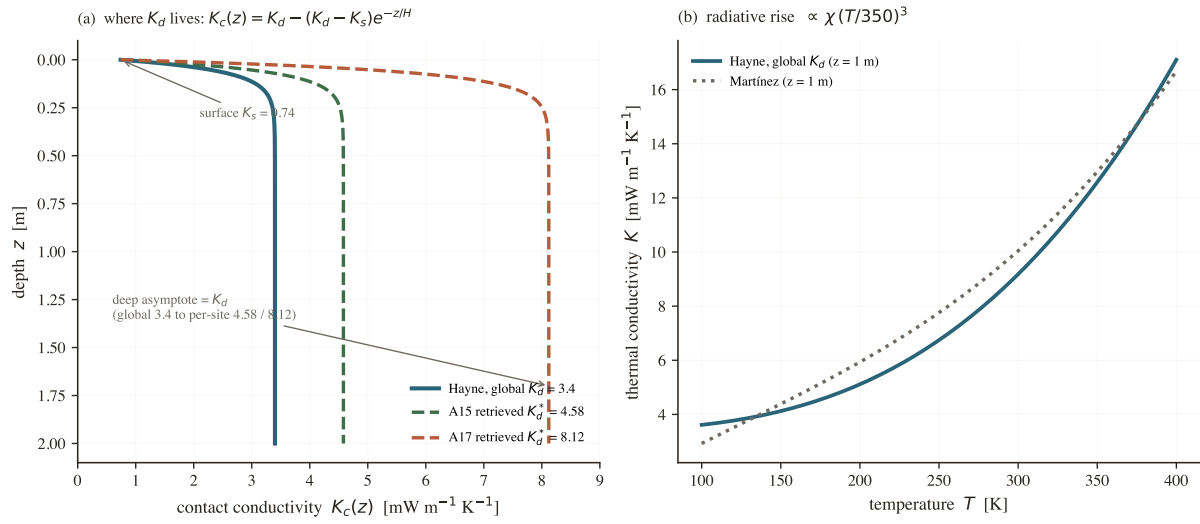


Figure 3.1: (a) Hayne contact conductivity $K_c(z)$: it rises from K_s to its deep asymptote K_d . The published global value is 3.4; our retrievals move it to 4.58 (A15) and 8.12 (A17). (b) Full $K(T)$ at 1 m for both models — the radiative term roughly doubles K by midday.

Chapter 4

The numerical solver, line by line

We cannot solve Eq. (2.2) by hand, so we discretise: chop the column into cells and time into steps, and advance the temperature numerically (Fig. 4.1).

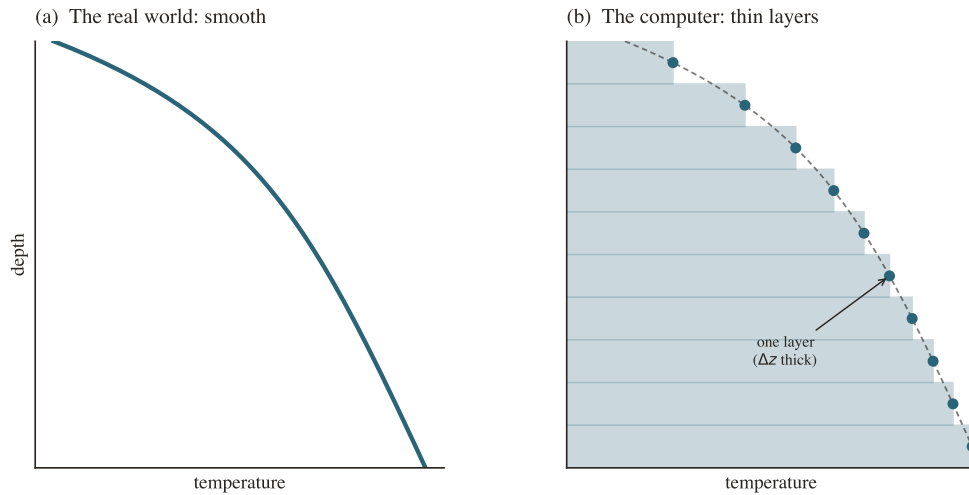


Figure 4.1: Discretising the column: cells in depth, steps in time; each step solves a small linear system that couples each cell to its neighbours.

4.1 The geometric depth grid

The daily wave needs millimetre resolution at the top, but the deep column can be coarse. So the cells grow geometrically with depth — each $\sim 8\%$ thicker than the one above:

```
def make_geometric_grid(z_max=5.0, dz0=0.002, growth=0.08):
    faces, dz = [0.0], dz0
    while faces[-1] < z_max:
        faces.append(faces[-1] + dz)
        dz *= (1 + growth) # each layer ~8% thicker
    # cell centres z_mid, thicknesses dz follow from the faces
```

This gives ~ 69 cells: fine where it matters, cheap where it does not.

4.2 One time step: Crank–Nicolson

We use the Crank–Nicolson scheme (second-order accurate, unconditionally stable). It averages the heat flow at the old and new time levels, producing a tridiagonal linear system $A T^{n+1} = d$ for the new temperatures. With face conductances α_i^L, α_i^R (from harmonic-mean face conductivities and the cell heat capacity $c_i = \rho_i c_{p,i} \Delta z_i$):

```
K = K_func(T_prev, z_mid) # conductivity at current T
Kf = harmonic_mean(K) # conductivity at cell faces
cap = rho * cp_func(T_prev) * dz # heat capacity of each cell
aL = dt * Kf[:-1] / (dz_c[:-1] * cap) # left-face conductance
aR = dt * Kf[1:] / (dz_c[1:] * cap) # right-face conductance
a = -0.5*aL; c = -0.5*aR; b = 1 + 0.5*(aL + aR) # the matrix A
```

```
d = 0.5*aL*Tleft + (1 - 0.5*(aL+aR))*T_prev + 0.5*aR*Tright # the RHS
```

(Full version: lunar/solver.py, function `_step`.)

4.3 Solving the system: the Thomas algorithm

A tridiagonal system is solved in one downward sweep and one upward sweep — fast and exact:

```
for i in range(1, n): # forward elimination
    m = b[i] - a[i]*cp[i-1]
    cp[i] = c[i] / m
    dp[i] = (d[i] - a[i]*dp[i-1]) / m
x[n-1] = dp[n-1]
for i in range(n-2, -1, -1): # back-substitution
    x[i] = dp[i] - cp[i]*x[i+1]
```

4.4 The non-linear surface: Newton's method

The surface energy balance Eq. (2.3) has a T_s^4 , so we cannot solve for T_s directly. We use Newton's method: guess T_s , compute how far the balance is from zero (R), and its slope (R'), and step:

```
for _ in range(max_iter):
    R = (1-A)*S - eps*sigma*T_s^4 - Ksurf*(Ts - Tsub)/(0.5*dz0)
    dR = -4*eps*sigma*T_s^3 - 2*Ksurf/dz0
    Ts = Ts - R/dR # Newton update; converges in a few steps
```

4.5 Stepping through a lunation: the spin-up loop

To find the steady state the brute-force way, we step through one lunation (~ 709 hourly steps), compare to the previous lunation, and repeat:

```
for cycle in range(1, n_lunations_spinup + 1):
    T_start = T.copy()
    for k in range(1, n_t): # march through one lunar day
        T = step(T, ...)
    if max(abs(T - T_start)) < 0.01 and cycle >= 2:
        break # "converged" <-- the trap (see Ch.5)
```

This works — but *how many* lunations is the whole problem of the next chapter.

Reaching steady state: the old method, the bug, and the fix

5.1 Why we even need steady state

We compare modelled deep temperatures to the Apollo sensors. The sensors recorded a *settled* regolith (we keep only the late, stable part of the record). And K_d is a *steady-state property*: its fingerprint is the settled deep gradient $d\langle T\rangle/dz = Q_b/K_d$ (Eq. (2.4)) — a gentler slope for higher K_d . Before the model settles, its deep gradient is still forming and reflects the starting guess, not K_d . So we can only read K_d off a settled profile.

In plain words

Judging K_d from an un-settled model is like judging how well a wall insulates by watching the temperature across it *while it is still warming up*: the slope is changing for reasons that have nothing to do with the insulation. You must wait for it to steady — or compute the steady state directly.

5.2 The old method and the bug we found (flag F1)

The original code reached steady state by a *fixed* 30-lunation spin-up (§4). **The symptom of trouble:** the retrieved K_d changed when we changed the starting temperature — a physical answer must not. The diagnosis: the column relaxes on the diffusive timescale of its full depth,

$$\tau \sim z_{\max}^2/\kappa \approx \text{order } 10^3 \text{ lunations}, \quad (5.1)$$

so after 30 lunations the deep regolith still holds its initial value (Fig. 5.1). Worse, the convergence test *lied*: because the deep drift is so slow, the cycle-to-cycle change is already < 0.01 K while the profile is far from settled. The sensors at 0.8–2.4 m sit exactly in the unsettled zone, so the starting guess leaked straight into K_d .

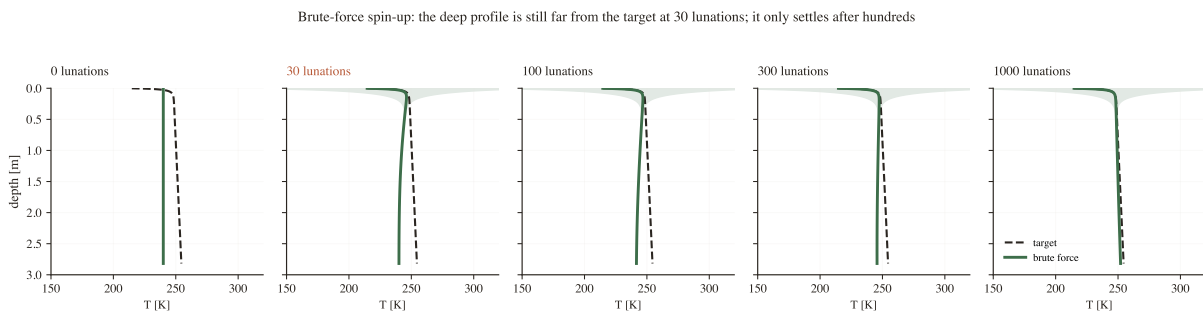


Figure 5.1: The old method (animated in `spinup.gif`). Green = current profile, dashed = true steady state, shaded = daily swing. At 30 lunations only the top ~ 0.5 m has moved; the deep part still sits at the start. It takes hundreds of lunations to settle.

★ FOR TOMORROW'S TALK

This is the crux of the “what went wrong” story. Show `spinup.gif`: the heat visibly fails to reach the deep sensors in 30 lunations, so the old K_d was biased by the starting guess. One sentence: “*a fixed short spin-up never settles the deep regolith, so the answer depended on where we started.*”

5.3 The fix: a flux-anchored shortcut

The escape is to use Eq. (2.4) instead of waiting for it. The new solver alternates two moves (Fig. 5.2, animated in `newmethod.gif`):

Step A — settle the skin. A *short* spin-up (a few lunations) settles only the fast surface skin and reads the cycle-mean “anchor” temperature just below it.

Step B — reconstruct the deep profile. From the anchor, integrate $d\langle T \rangle/dz = Q_b/K$ straight down. This *computes* the deep profile in one shot instead of waiting for heat to diffuse there.

The reconstruction is the surprising part — the deep temperatures are not simulated, they are walked out from the anchor along the known slope:

```
for i in range(i0, N-1): # walk downward from the anchor depth
    q = Q_b - u[i] # u = small rectification correction
    Ki = K(T[i], z[i])
    Th = T[i] + 0.5*q/Ki*dz # midpoint (RK2) for accuracy
    T[i+1] = T[i] + q/K(Th, z_half)*dz # dT/dz = (Q_b - u)/K
```

With $Q_b = 21 \text{ mW m}^{-2}$ and $K \approx 9 \text{ mW m}^{-1} \text{ K}^{-1}$ at depth, the slope is $\sim 2.3 \text{ K/m}$, so each metre below the anchor adds $\sim 2.3 \text{ K}$ (animated in `reconstruct.gif`).

The new method reaches the SAME target in a few steps — the ‘reconstruct’ move snaps the deep profile straight onto the steady gradient

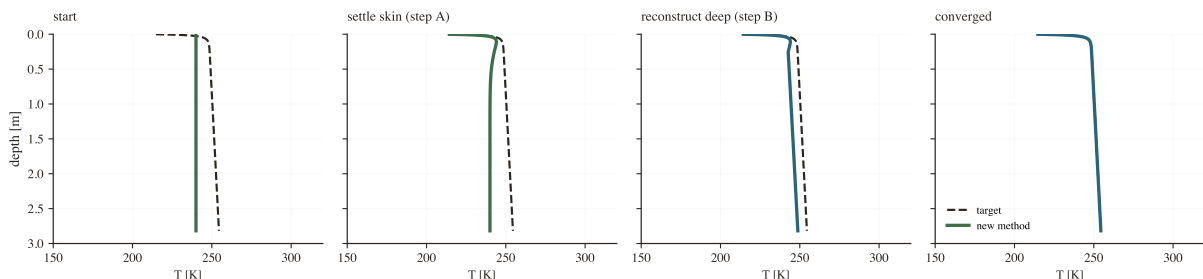


Figure 5.2: The new method (animated in `newmethod.gif`). Step A settles the skin; Step B’s reconstruct move snaps the deep profile onto the steady gradient; a dozen short rounds refine it to the target.

The two moves are iterated in a short two-stage loop:

```
for stage in (coarse_anchor_0.25m, fine_anchor_0.55m):
    for it in range(stage.max_iters):
        out = solve_pixel(..., n_lunations_spinup=stage.n_inner) # Step A
        Tm = out.T.mean(axis=1) # cycle mean
        u = rectified_flux(out.T, z, K)
        T = reconstruct(Tm, z, stage.anchor, Q_b, K, u) # Step B
        if abs(Tm[anchor] - prev) < stage.tol and it >= 2: break # drift check
        prev = Tm[anchor]
```

In total this uses ~ 150 lunations of stepping ($\sim 9 \text{ s}$), versus the $\sim 1000+$ lunations the old method must crawl through.

5.4 They give the same answer

The decisive check: take the *old* method and just run it longer. As the lunations increase, its profile marches onto the new method's answer (Fig. 5.3); started from two very different temperatures (240 K and 260 K) it lands on the same curve (Fig. 5.4). They must — both enforce the same two steady-state conditions, and the solution is unique. The new method just reaches it without waiting.

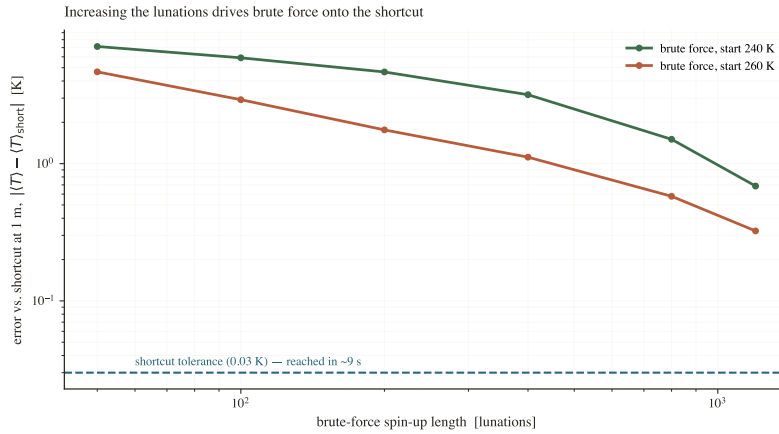


Figure 5.3: Brute-force error vs. the new method falls steadily as the spin-up lengthens; two starts converge to the same value.

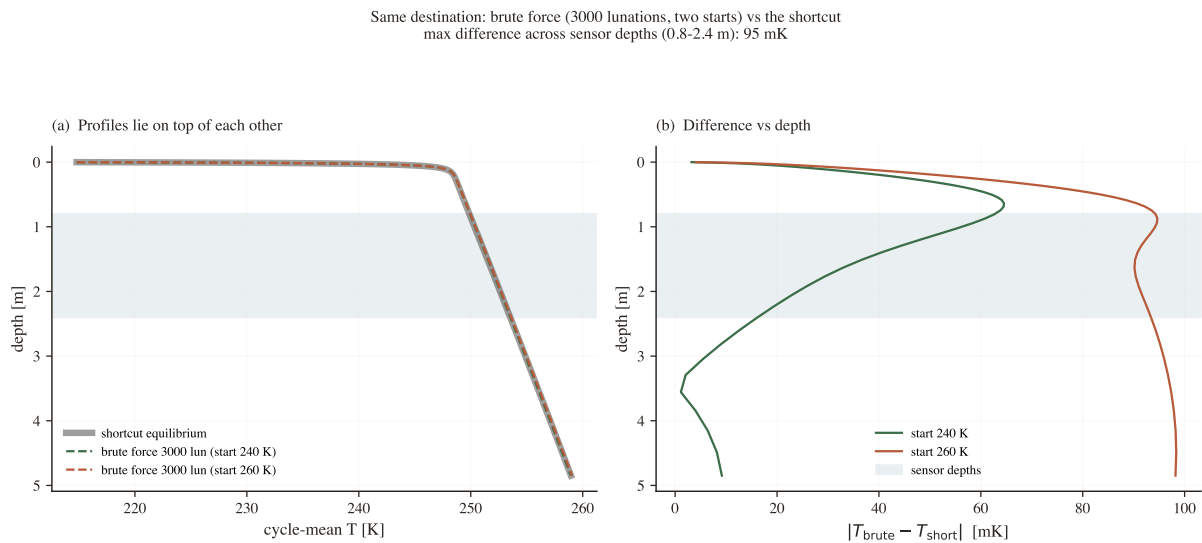


Figure 5.4: Brute force run to 3000 lunations from 240 K and 260 K (dashed) lies on top of the new method (solid): same profile to tens of mK.

★ FOR TOMORROW'S TALK

The single most convincing slide. Two very different starting guesses, plus the fast method, all land on one curve (Fig. 5.4). Say: “*the new method isn’t a different model — it reaches the exact same steady state, just $\sim 30\times$ faster, and it certifies that it got there.*”

5.5 It certifies itself

Every solve returns two numbers proving convergence: the anchor *drift* between iterations (< 0.03 K; typically 3 mK) and the *flux closure* $\max |\langle q \rangle - Q_b|/Q_b$ below the anchor ($< 2\%$; Fig. 5.5). These are exactly the check the old fixed spin-up could never pass.

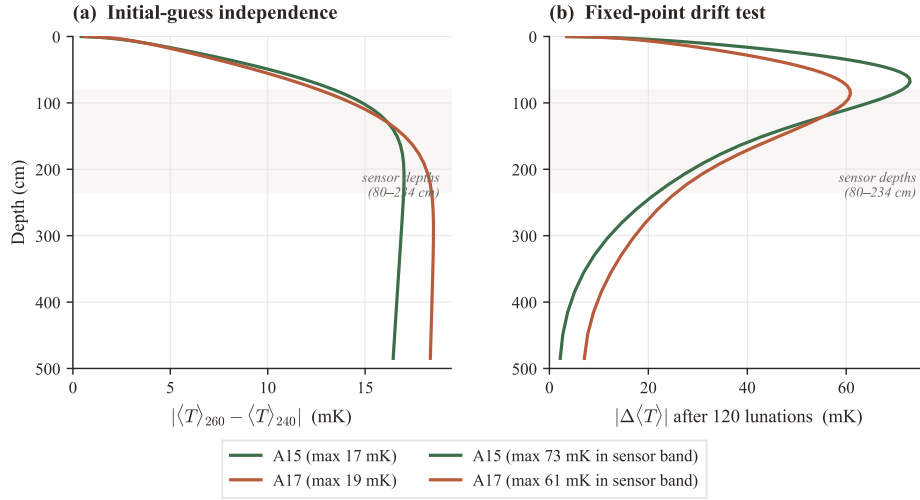


Figure 5.5: Guess-independence and flux closure — the returned proof that a solve reached the steady state.

Retrieving K_d from the data

6.1 One forward run: `run_with`

The hub of the pipeline turns one trial K_d into one settled profile:

```
def run_with(site, kd):
    grid = make_geometric_grid(^GRID)
    insol = (1 - site.albedo) * S0 * solar_cosine(site) # sunlight forcing
    K = lambda T, z: conductivity_hayne(T, z, Kd=kd) # Hayne with this K_d
    eq = solve_periodic_equilibrium(grid, t, insol, Q_b=site.Q_b, K_func=K, ...)
    return grid.z_mid, eq.T_mean # depths and the settled mean profile
```

6.2 The sweep and the minimum

We try a grid of K_d values; for each, `run_with` gives a settled profile, and we measure the misfit to the deep sensors,

$$\text{RMSE}(K_d) = \sqrt{\frac{1}{N} \sum_i [\langle T \rangle(z_i; K_d) - T_i^{\text{obs}}]^2}. \quad (6.1)$$

The best-fit K_d^* is the minimum, refined by fitting a parabola through the three points around it:

```
rmse = np.sqrt((R[idx]^2).mean(axis=0)) # RMSE at each trial K_d
k = np.argmin(rmse) # nearest grid minimum
kd_star = parabola_vertex(kd_grid[k-1:k+2], rmse[k-1:k+2]) # sub-grid minimum
```

Each site shows a clean single minimum (Fig. 6.1).

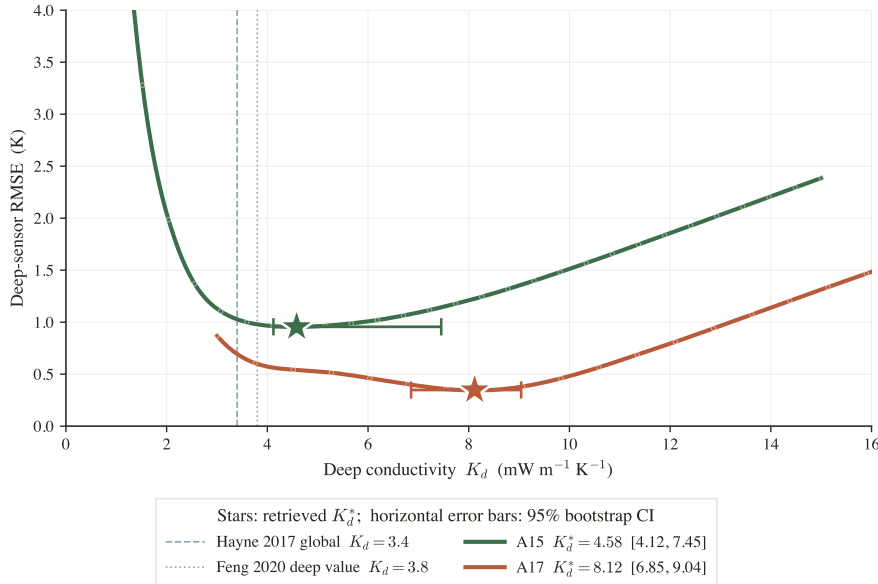


Figure 6.1: Deep-sensor RMSE vs. trial K_d . The minimum is K_d^* ; Apollo 17's sits at a clearly higher conductivity than Apollo 15's.

6.3 The result

$$K_{d,A15}^* = 4.58, \quad K_{d,A17}^* = 8.12 \text{ mW m}^{-1} \text{ K}^{-1},$$

Apollo 17 about twice Apollo 15. With these the modelled deep temperatures track the data far better than the global value (Fig. 6.2); at Apollo 17 the misfit roughly halves.

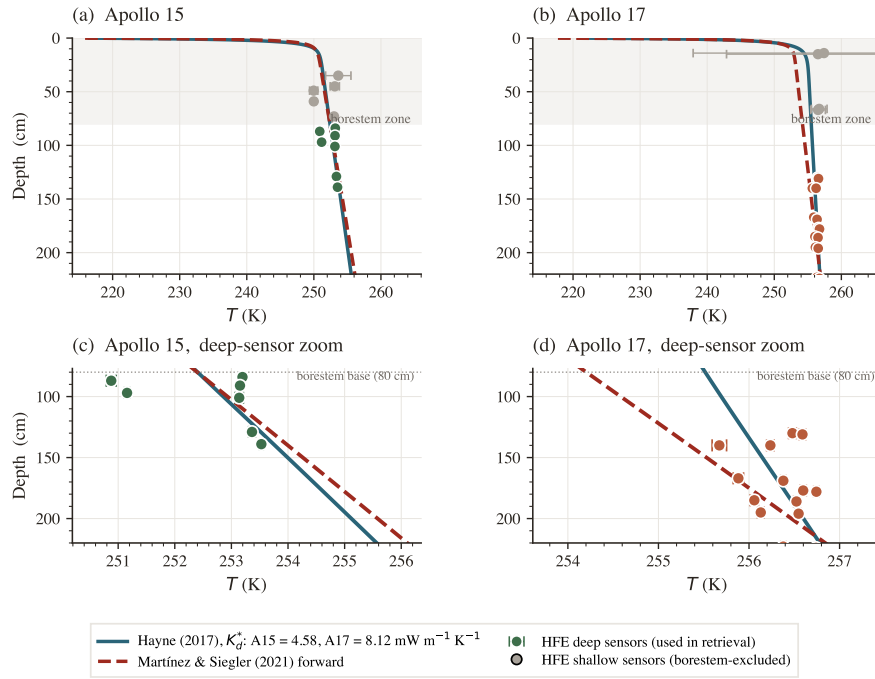


Figure 6.2: Settled mean temperature vs. depth using each site's retrieved K_d^* , against the deep-sensor measurements.

How sure are we? (uncertainty)

7.1 The bootstrap

To get an error bar without assuming a noise model, we *resample*: draw the sensors with replacement (and jitter their depths by ± 2.5 cm), re-fit K_d^* , and repeat 1500 times. The spread of those 1500 fits is the uncertainty (Fig. 7.1). The 95% intervals are [4.12, 7.45] (A15) and [6.85, 9.04] (A17); the inter-site contrast is 3.31 with $p \approx 0.011$ — about a 1% chance of being a fluke.

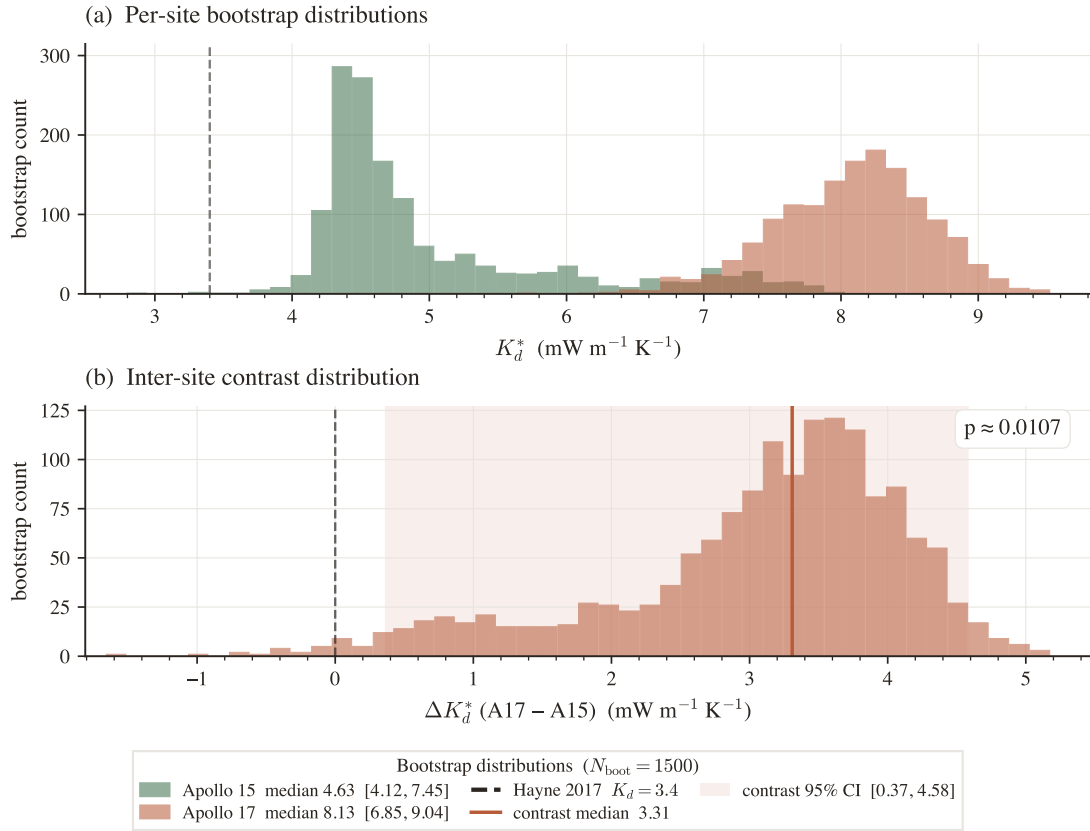


Figure 7.1: Bootstrap distributions of K_d^* . The two humps barely overlap — the contrast is statistically real.

7.2 Bayesian cross-check and model selection

An independent Bayesian sampler (Fig. 7.3 explains the idea) maps the joint posterior in (K_d, Q_b) ; even with Q_b uncertain, Apollo 17 stays more conductive $\sim 83\%$ of the time. A model comparison (AICc; Fig. 7.3) prefers “different K_d per site” over “one shared K_d ” — so the data favour a *conductivity* contrast, not a flux artefact. (These are summarised here; the letter carries the full treatment.)

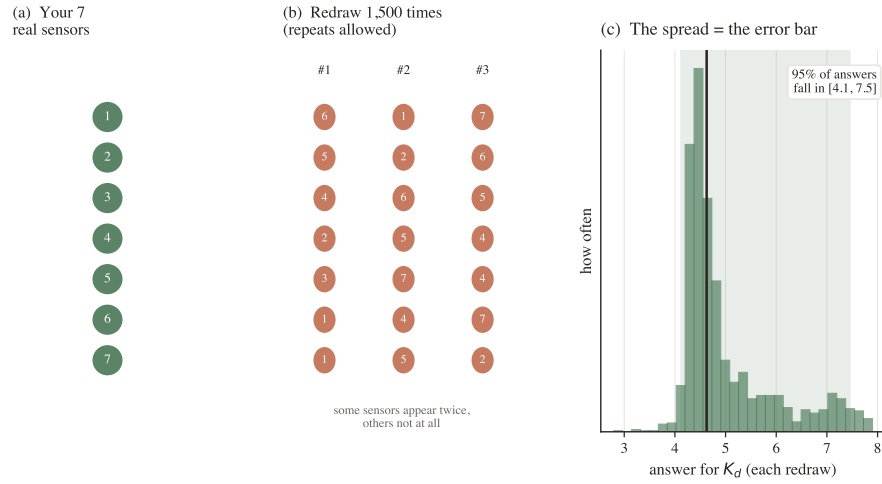


Figure 7.2: What the bootstrap does, schematically: resample the data many times, re-fit each time, read the spread.

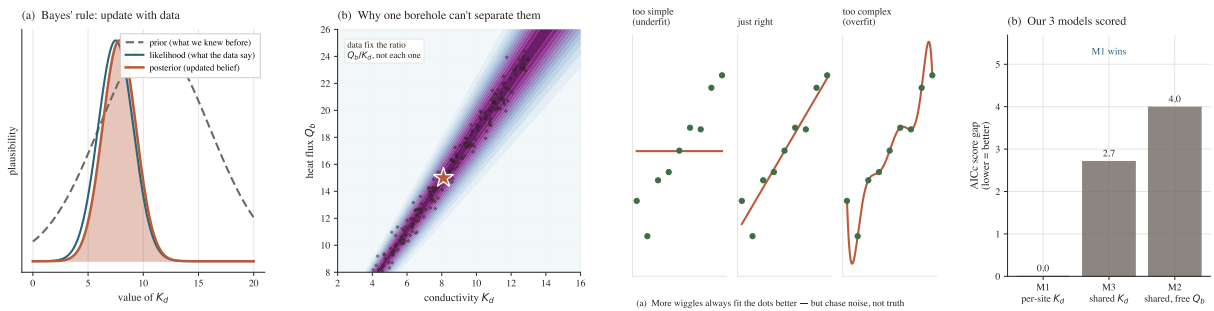


Figure 7.3: Left: the idea of MCMC posterior sampling. Right: AICc model selection trades fit quality against the number of free parameters.

Chapter 8

The role of the basal flux Q_b

K_d cannot be read in isolation. The deep gradient is Q_b/K (Eq. (2.4)), so K_d and Q_b trade off: assume more heat from below and you infer a higher conductivity for the same temperatures (Fig. 8.1). This single record cannot separate them, so we *fix* Q_b at the published values (21 and 15 mW m^{-2}) and report K_d *conditional* on them. The *direction* of the contrast survives even when Q_b is allowed to vary, and Q_b is one of the largest single terms in the error budget — a direct fingerprint of this coupling.

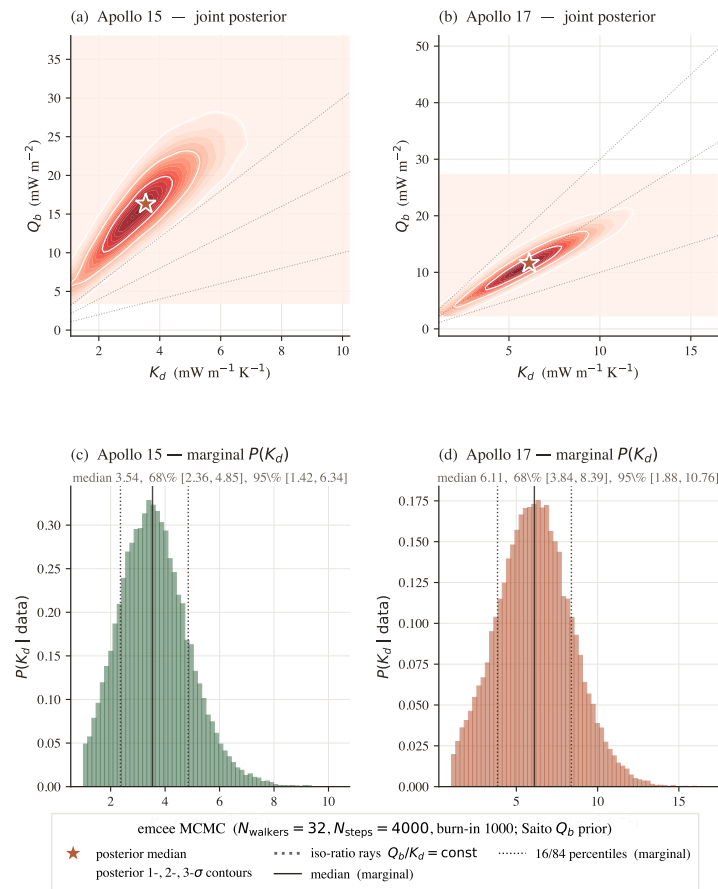


Figure 8.1: The K_d – Q_b trade-off: many combinations along a diagonal fit almost equally well, so K_d is conditional on Q_b . The contrast between sites is more robust than either absolute value.

★ FOR TOMORROW'S TALK

Pre-empt the obvious question. Say up front: “ K_d rides on the assumed basal flux Q_b — they trade off. So I report K_d given the published Q_b , and the robust claim is the direction: Apollo 17 is more conductive than Apollo 15.”

Why might the two sites differ?

A factor-of-two contrast is large but physically reasonable: the two boreholes sample different regolith.

	Apollo 15	Apollo 17
Setting	Hadley Rille (mare/front)	Taurus-Littrow valley
Bond albedo	0.131	0.137
Deepest sensor	1.39 m	2.34 m
Basal flux Q_b (fixed)	21 mW m ⁻²	15 mW m ⁻²
Retrieved K_d^*	4.58	8.12 mW m ⁻¹ K ⁻¹

Heat moves through dry regolith mainly by grain-to-grain contact, so a higher K_d points to denser, more compacted, or more rock-rich deep regolith. Plausible contributors at Apollo 17: (i) its material — highland massif debris plus dense pyroclastic dark-mantle deposits — versus Hadley’s mare-basalt regolith; (ii) independent support from the density-based Martínez model, which prefers a higher effective deep density at Apollo 17 (Ch. 10); (iii) more buried rock fragments or a more mature, sintered column. The honest framing: *Apollo 17’s regolith conducts heat about twice as efficiently as Apollo 15’s, conditional on the published basal fluxes.*

Independent cross-checks

Diviner surface closure. Run the model forward with the retrieved K_d and compare its predicted *surface* day–night curve to orbital Diviner data — a check that uses no heat-flow data. They agree (Fig. 10.1), so the deep fit and the surface are self-consistent.

Martínez α -sweep. Repeat the retrieval with the density-based model (Eq. (3.3)); each site prefers a different deep density — the same contrast through a different model (Fig. 10.1).

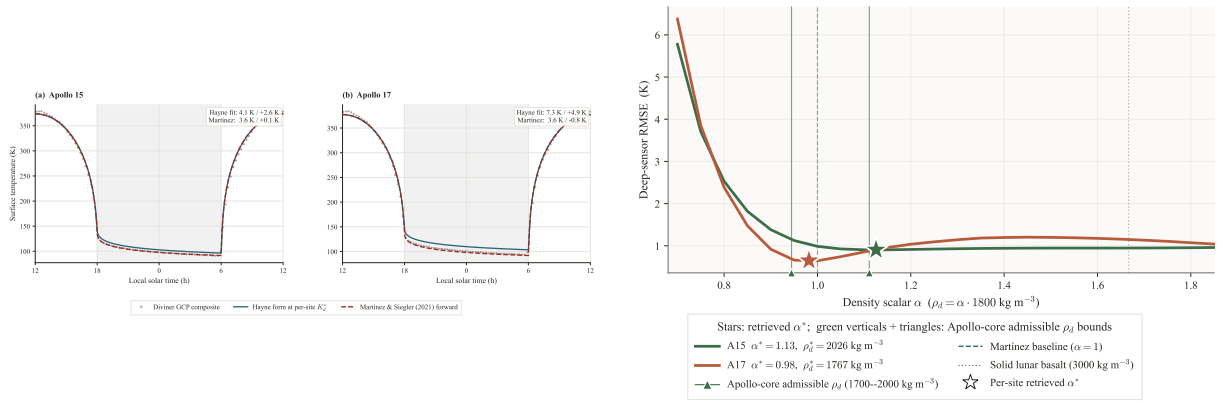


Figure 10.1: Left: the model with the retrieved K_d reproduces the Diviner surface temperatures. Right: the Martínez density retrieval tells the same story.

How the code is organised, and a C++ future

The code is layered (Fig. 11.1): a physics package `lunar/` (the models, grid, solver, equilibrium), thin driver scripts `scripts/`, and the documents. One function, `run_with`, is the hub everything calls.

The three-layer architecture

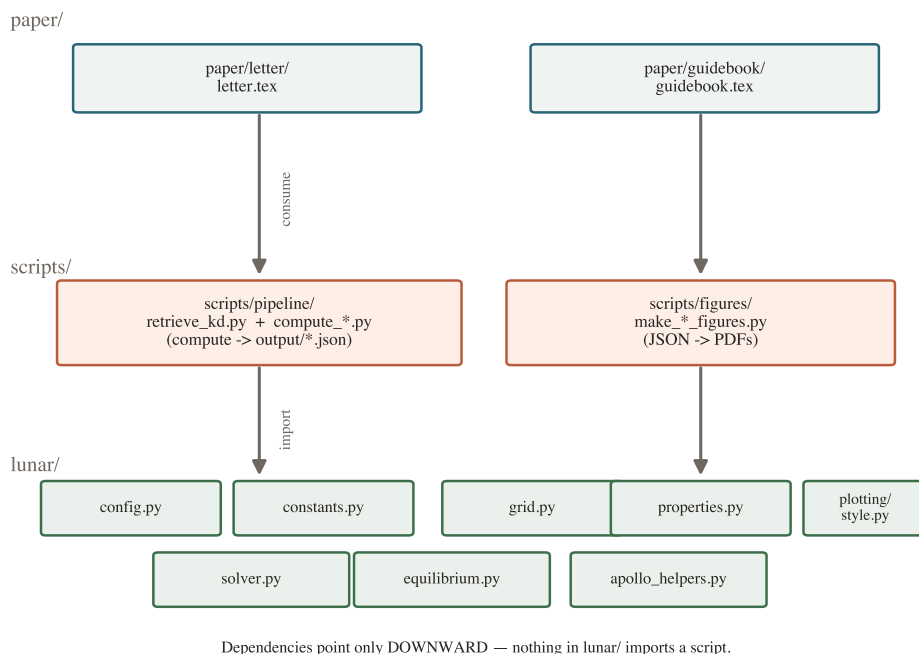


Figure 11.1: Three layers; imports point only downward. The numerics live in three small modules (`grid`, `solver`, `equilibrium`).

C++ port. The time is spent in the inner step (§4), run hundreds of thousands of times. The tridiagonal solve is already JIT-compiled; the matrix assembly is interpreted Python — exactly where a compiled language wins (~ 10 – $100\times$). The plan: port `grid`, `solver`, `equilibrium` to C++, keep configuration and analysis in Python, and validate the port against the existing 37-test suite and the committed steady-state profiles. The three speed-ups multiply: the algorithm (the shortcut, $\sim 30\times$, done) $\times$ compilation $\times$ parallel independent solves. Not needed for this paper; valuable at scale.

Conclusion: how to defend every claim

- **What we did.** Retrieved the deep regolith conductivity separately at Apollo 15 and 17: 4.58 and $8.12 \text{ mW m}^{-1} \text{ K}^{-1}$.
- **How.** A 1-D heat model (Ch. 2–4) with the Hayne conductivity (Ch. 3), solved to a certified steady state (Ch. 5), with K_d chosen to minimise the deep-sensor misfit (Ch. 6).
- **Why it's right.** The solver is unique by construction, certifies its own convergence, is guess-independent, and is confirmed by brute force converging onto it (§5.4); the result is statistically significant (Ch. 7) and echoed by an independent model (Ch. 10).
- **The honest caveat.** K_d is conditional on the assumed Q_b (Ch. 8); the robust statement is the inter-site *direction*.
- **The physics.** Apollo 17's deep regolith conducts heat about twice as efficiently — plausibly denser/more compacted/more rock-rich material (Ch. 9).

Reproduce everything: `make retrieve` (the result), `notebooks/equilibrium_demo.ipynb` (old vs. new), and `pytest tests/test_equilibrium.py` (the certification). The animations `spinup.gif`, `newmethod.gif`, `reconstruct.gif` accompany Chapters 5.